

Übung 11

a) abheben

Wie in der Vorlesung besprochen, ist die Methode `abheben(Geldbetrag)` noch nicht ganz ideal implementiert.

- Hier soll das Template-Method-Muster zum Einsatz kommen. Zeichnen Sie die beteiligten Klassen/Methoden in ein UML-Diagramm. Bitte beschränken Sie sich wirklich auf alles, was zur Template-Method gehört!
 - Denken Sie gut über die Sichtbarkeitsmodifizierer nach – jede public-Methode kann einfach so von überallher aufgerufen werden...
- Überlegen Sie und beantworten Sie folgende Frage: Welche spätere Änderung/Erweiterung wird in diesem Programm durch den Einsatz des Musters leicht?
- Setzen Sie die Umstrukturierung der `abheben`-Methode im Code um. Arbeiten Sie mit den drei konkreten Klassen Girokonto, Spargbuch und Aktienkonto.

Sie sollten bereits einen JUnit-Test für die Spargbuch-Klasse geschrieben haben. Wenn Sie alles korrekt implementiert haben, sollte er weiterhin problemlos laufen.

Fügen Sie zwei neue Klassen hinzu, die von Konto erben:

- `bankprojekt.basisdaten.Kinderkonto`: Man darf höchstens 50,-€ abheben und dabei natürlich nicht den Kontostand überziehen.
- `bankprojekt.basisdaten.Festgeldkonto`: Abhebungen sind nur bis höchstens zu dem Betrag erlaubt, der vorher gekündigt worden ist. Noch einmal kann man den gekündigten Betrag natürlich nicht abheben.
 - Fügen Sie eine zusätzliche Methode


```
public void kuendigen(Geldbetrag betrag)
```

 hinzu, die den genannten `betrag` als gekündigt markiert (höchstens jedoch den Kontostand)

b) Konto erstellen

Die Bank hat zwei Methoden `girokontoErstellen()` und `spargbuchErstellen()`. Jede davon ist vor allem dafür zuständig, das jeweilige Konto-Objekt zu erzeugen und in der Bank-Verwaltung zu speichern, vermischt also Erzeugung und Arbeit mit dem Objekt. Außerdem gab es da noch `mockEinfuegen()` – eine Methode, die irgendwie nur zum Testen da war, aber genauso aufgebaut war...

Verwenden Sie hier das Abstract-Factory-Muster, um das zu ändern:

Ersetzen Sie die drei genannten Methoden in der Bank durch eine Methode

- ```
public long kontoErstellen(Kontofabrik fabrik, ...)
```

 (... soll für die von Ihnen benötigten Parameter stehen)  
 Die Klasse `bankprojekt.fabriken.Kontofabrik` und ggf. weitere Unterklassen müssen Sie dazu ergänzen.
- Überlegen Sie und beantworten Sie folgende Frage: Welche spätere Änderung/Erweiterung wird in diesem Programm durch den Einsatz des Musters leicht?
- Ändern Sie außerdem Ihren Bank-Test aus Übung 6 ab: Löschen Sie die Methode `mockEinfuegen()` der Bank. Nutzen Sie stattdessen die neue Methode `kontoErstellen`.

Erzeugen Sie wieder mit dem Maven-assembly-Plugin das jar-Archiv mit den Quelltexten und der pom.xml geben Sie es ab. Vergessen Sie auch das UML-Diagramm und die Antworten auf die beiden Fragen nicht!